🏛 GLA UNIVERSITY, MATHURA — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

HPC Masterclass | Day 3 Pre-Assessment Question Paper

Course: B.Tech CSE 7th Sem — High Performance Computing Masterclass
Module: Day 3 Baseline Readiness — Deep Learning Systems, Precision, Hardware Architecture & Cluster Concepts
Duration: 60 Minutes | **Total Questions**: 50 MCQs | **Maximum Marks**: 50

Student Name: _____ **Roll Number**: _____ **Section**: _____

📝 Instructions for Candidates

1. All questions are compulsory. Each question carries exactly 1 mark.
2. Select the single best answer for each question.
3. No negative marking.

Q01. In standard deep learning training, what is the primary computational purpose of the backward pass (backpropagation)?

- ▢ **(a)** To compute predicted outputs from input features.
- ▢ **(b)** To evaluate gradients of the scalar loss with respect to all trainable parameters via the chain rule of calculus.
- ▢ **(c)** To convert 32-bit floating-point tensors into 8-bit integers.
- ▢ **(d)** To schedule worker threads on available CPU cores.

Q02. What is the key difference between Eager Execution and Static Computational Graphs in deep learning frameworks like TensorFlow?

- ▢ **(a)** Eager execution requires writing all code in C++, whereas static graphs use Python.
- ▢ **(b)** Eager execution evaluates operations immediately and imperatively, while static graphs construct a declarative symbolic DAG (Directed Acyclic Graph) before execution.
- ▢ **(c)** Static graphs do not support GPU acceleration.
- ▢ **(d)** Eager execution completely disables backpropagation.

Q03. In TensorFlow 2.x, which Python decorator transforms a dynamic Python function into a high-performance compiled static TensorFlow graph?

- ▢ **(a)** @tf.function
- ▢ **(b)** @tf.compile_graph
- ▢ **(c)** @tf.eager_mode
- ▢ **(d)** @tf.static_run

Q04. Why is mini-batch Stochastic Gradient Descent (SGD) preferred over full-batch Gradient Descent when training on large datasets?

- ▢ **(a)** Full-batch GD requires loading the entire dataset into memory at once, which exceeds system RAM, and offers fewer gradient updates per epoch.
- ▢ **(b)** Full-batch GD is mathematically incapable of minimizing convex loss functions.
- ▢ **(c)** Mini-batch SGD disables all multi-threading on modern processors.
- ▢ **(d)** Mini-batch SGD guarantees zero variance in gradient estimates.

Q05. How does the Adam optimizer calculate parameter updates compared to basic Stochastic Gradient Descent (SGD)?

- ▢ **(a)** Adam sets all weights to zero at the end of each epoch.
- ▢ **(b)** Adam maintains per-parameter adaptive learning rates by tracking both first moments (mean) and uncentered second moments (variance) of the gradients.
- ▢ **(c)** Adam computes second-order Hessian matrices directly using numerical differentiation.
- ▢ **(d)** Adam updates only the bias terms and ignores weight matrices.

Q06. What is the primary motivation for applying Batch Normalization layers in deep neural network architectures?

- ▢ **(a)** To replace all activation functions with identity operations.
- ▢ **(b)** To stabilize optimization by normalizing layer inputs across the batch dimension to zero mean and unit variance, mitigating internal covariate shift.
- ▢ **(c)** To reduce the physical storage size of the dataset on disk.
- ▢ **(d)** To encrypt model weights during training.

Q07. What happens to the spatial dimensions (height and width) of a 32×32 image when processed by a 3×3 2D convolution with stride 1 and 'VALID' padding?

- ▢ **(a)** They expand to 34×34.
- ▢ **(b)** They shrink to 30×30.
- ▢ **(c)** They remain exactly 32×32.
- ▢ **(d)** They drop to 16×16.

Q08. What causes the 'vanishing gradient problem' in deep neural networks using Sigmoid or Tanh activations?

- ▢ **(a)** The hardware runs out of floating-point registers.
- ▢ **(b)** The derivative of Sigmoid is bounded between 0 and 0.25, causing repeated multiplication across many layers in backpropagation to exponentially decay gradients toward zero.
- ▢ **(c)** The learning rate is automatically multiplied by 100 after each layer.
- ▢ **(d)** The input dataset has too many training features.

Q09. What role does the 'Dropout' regularization technique play during deep learning training?

- ▢ **(a)** It deletes corrupt image files from the training directory.
- ▢ **(b)** It randomly zeroes out a fraction p of neuron activations during each forward pass, preventing complex co-adaptations between neurons.

- ▢ **(c)** It drops slow worker nodes from the distributed cluster.
- ▢ **(d)** It reduces the precision of all model parameters to 8-bit integers.

Q10. In deep learning frameworks, why must intermediate layer activations computed during the forward pass be cached in memory?

- ▢ **(a)** To allow the operating system to verify user credentials.
- ▢ **(b)** Because the partial derivatives computed during backpropagation require the activation values from the forward pass as multipliers.
- ▢ **(c)** To prevent the GPU from entering sleep mode.
- ▢ **(d)** To display real-time training images on the monitor.

Q11. How many bits are used for the exponent and mantissa (fraction) in standard IEEE 754 single-precision floating point (FP32)?

- ▢ **(a)** 1 sign bit, 8 exponent bits, 23 mantissa bits.
- ▢ **(b)** 1 sign bit, 5 exponent bits, 10 mantissa bits.
- ▢ **(c)** 1 sign bit, 11 exponent bits, 52 mantissa bits.
- ▢ **(d)** 1 sign bit, 8 exponent bits, 7 mantissa bits.

Q12. What is the structural bit-level difference between standard FP16 and Google/Intel Bfloat16 (BF16)?

- ▢ **(a)** FP16 uses 16 bits, while BF16 uses 32 bits.
- ▢ **(b)** FP16 has 5 exponent bits and 10 mantissa bits, whereas BF16 has 8 exponent bits and 7 mantissa bits (matching FP32's dynamic range).
- ▢ **(c)** BF16 cannot represent negative numbers.
- ▢ **(d)** FP16 is only used on CPUs, while BF16 is only used on hard drives.

Q13. Why does mixed-precision training (using FP16 or BF16 for matrix operations alongside FP32 master weights) provide major speedups on modern HPC accelerators?

- ▢ **(a)** It halves memory bandwidth pressure, doubles tensor core compute throughput, and reduces cache footprints by 50%.
- ▢ **(b)** It eliminates the need for backpropagation entirely.
- ▢ **(c)** It automatically converts deep neural networks into decision trees.
- ▢ **(d)** It bypasses the operating system kernel network stack.

Q14. In the context of post-training quantization, what is INT8 quantization?

- ▢ **(a)** Compressing weights and activations from 32-bit floats into 8-bit signed integers using scale and zero-point parameters.
- ▢ **(b)** Deleting 8 out of every 10 layers in a deep neural network.
- ▢ **(c)** Rounding all floating-point numbers to zero.
- ▢ **(d)** Running neural networks on an 8-bit Intel 8085 microprocessor.

Q15. What is the mathematical definition of 'Arithmetic Intensity' (Operational Intensity) in the Roofline Model?

- ▢ **(a)** Total Wall Clock Time / Total CPU Cores

- ▢ **(b)** Total Floating-Point Operations (FLOPs) / Total Bytes transferred from main memory (FLOPs/Byte)
- ▢ **(c)** Dataset Size in Megabytes / Batch Size
- ▢ **(d)** Network Packet Size / Clock Frequency

Q16. According to the Roofline Model, what fundamental hardware characteristic limits the maximum throughput of an operation with very low arithmetic intensity?

- ▢ **(a)** Peak theoretical FLOPS of the vector ALUs.
- ▢ **(b)** Memory bandwidth (DRAM or HBM transfer rate in GB/s or TB/s).
- ▢ **(c)** Hard drive seek latency.
- ▢ **(d)** Compiler optimization flags.

Q17. Which of the following neural network operations possesses the HIGHEST arithmetic intensity?

- ▢ **(a)** Elementwise Vector Addition ($y = a + b$)
- ▢ **(b)** ReLU Activation ($y = \max(0, x)$)
- ▢ **(c)** General Matrix Multiplication (GEMM, $C = A \times B$) with large square dimensions ($N > 2048$)
- ▢ **(d)** Layer Normalization

Q18. What is the purpose of 'Loss Scaling' when performing mixed-precision training with IEEE FP16?

- ▢ **(a)** To artificially multiply the model loss by a scale factor to prevent small gradient values from underflowing to zero in 16-bit representation.
- ▢ **(b)** To divide the loss to ensure training finishes in fewer epochs.
- ▢ **(c)** To convert the loss value into an integer percentage.
- ▢ **(d)** To normalize the dataset images to mean zero.

Q19. What is the primary trade-off encountered when applying weight pruning to a deep neural network?

- ▢ **(a)** Pruning guarantees zero loss in accuracy regardless of sparsity percentage.
- ▢ **(b)** Unstructured pruning creates irregular sparse matrices that require specialized sparse accelerators to achieve actual wall-clock speedups compared to dense GEMMs.
- ▢ **(c)** Pruning doubles the memory footprint of the model on disk.
- ▢ **(d)** Pruning converts the neural network into an uncompressed audio waveform.

Q20. What does a dynamic range underflow error mean during floating-point matrix multiplication?

- ▢ **(a)** The result is larger than the maximum positive number representable, becoming positive infinity.
- ▢ **(b)** The magnitude of a non-zero number is smaller than the smallest subnormal representable number, causing it to be flushed to zero.
- ▢ **(c)** The CPU registers have overheated.
- ▢ **(d)** The input tensor contains negative numbers.

Q21. What is the typical access latency of CPU L1 Data Cache compared to Main Memory (DRAM)?

- ▢ **(a)** L1 cache is ~1 nanosecond (4-5 cycles), while DRAM is ~60-80 nanoseconds (200+ cycles).
- ▢ **(b)** L1 cache and DRAM have identical latency.

- ▢ **(c)** L1 cache is slower than DRAM by a factor of 10.
- ▢ **(d)** DRAM takes 5 picoseconds, while L1 cache takes 1 second.

Q22. What is the standard hardware cache line size on modern x86_64 processors?

- ▢ **(a)** 4 bytes
- ▢ **(b)** 16 bytes
- ▢ **(c)** 64 bytes
- ▢ **(d)** 4096 bytes

Q23. What does SIMD stand for in computer architecture, and how does it benefit deep learning tensor computations?

- ▢ **(a)** Single Instruction Multiple Data; it executes one operation (e.g., FMA) simultaneously across multiple data elements in wide vector registers.
- ▢ **(b)** Serial Input Multi Device; it allows one mouse to control several servers.
- ▢ **(c)** Synchronous Instruction Memory Dispatch; it compiles Python to assembly.
- ▢ **(d)** Shared Interface for Memory Devices; it formats hard drives.

Q24. How wide are Intel AVX-512 vector registers (ZMM0 to ZMM31), and how many 32-bit floating-point numbers can one register hold?

- ▢ **(a)** 128 bits wide, holding 4 floats.
- ▢ **(b)** 256 bits wide, holding 8 floats.
- ▢ **(c)** 512 bits wide, holding 16 single-precision (FP32) floats.
- ▢ **(d)** 1024 bits wide, holding 64 floats.

Q25. Why is the PCIe bus frequently a major performance bottleneck during deep learning training on discrete GPU or accelerator nodes?

- ▢ **(a)** Because PCIe bandwidth (e.g., ~64 GB/s for PCIe Gen5 x16) is drastically lower than accelerator on-chip HBM bandwidth (up to 2000-3000 GB/s).
- ▢ **(b)** Because PCIe cannot transmit binary floating-point numbers.
- ▢ **(c)** Because PCIe operates strictly at 10 megabits per second.
- ▢ **(d)** Because the PCIe bus only operates when the operating system is shutting down.

Q26. In a Non-Uniform Memory Access (NUMA) multi-socket server, what is 'remote memory access latency'?

- ▢ **(a)** Accessing files from a remote cloud server in another country.
- ▢ **(b)** The additional latency incurred when a CPU core accesses DRAM physically attached to a different CPU socket across the inter-socket interconnect (UPI/QPI/Infinity Fabric).
- ▢ **(c)** The time it takes to wake up a hard drive from power-saving mode.
- ▢ **(d)** Accessing registers in an adjacent thread.

Q27. What is High Bandwidth Memory (HBM2e / HBM3), and how is it physically packaged in modern HPC accelerators (e.g., Intel GPU Max, Gaudi, NVIDIA H100)?

- ▢ **(a)** Standard DDR5 DIMMs plugged into motherboard slots.

- ▢ **(b)** Vertically stacked DRAM dies connected through Through-Silicon Vias (TSVs) placed directly on the same silicon interposer adjacent to the compute chip.
- ▢ **(c)** USB flash drives configured in a RAID-0 array.
- ▢ **(d)** Solid-state drives connected via SATA cables.

Q28. What is the purpose of an input data pipeline prefetching mechanism (e.g., 'dataset.prefetch(tf.data.AUTOTUNE)') in deep learning?

- ▢ **(a)** To download the next day's weather forecast.
- ▢ **(b)** To overlap CPU data preprocessing (decoding, augmentation) for step $N+1$ with accelerator compute execution for step N, preventing compute starvation.
- ▢ **(c)** To convert the training labels from strings to integers.
- ▢ **(d)** To permanently store intermediate gradients on hard disk.

Q29. What is a 'memory leak' in Python deep learning code, and how does it typically manifest during long-running training loops?

- ▢ **(a)** Physical silicon melting due to high voltage.
- ▢ **(b)** Tensors or Python objects accumulating references in global scope (e.g., appending loss tensors to a list without calling '.item()'), leading to gradual exhaustion of RAM and an Out-Of-Memory (OOM) crash.
- ▢ **(c)** The CPU cooling fans slowing down.
- ▢ **(d)** Model weights becoming NaN (Not-a-Number).

Q30. Why is 'data layout transformation' (e.g., converting NHWC to NCHW or vice versa) an expensive operation for GPUs and CPUs?

- ▢ **(a)** Because it requires training a new neural network.
- ▢ **(b)** Because it performs a non-contiguous memory transposition that requires strided reads and writes across DRAM, which underutilizes memory bus bursts and thrashes cache lines.
- ▢ **(c)** Because operating systems do not allow 4D arrays.
- ▢ **(d)** Because it doubles the numerical value of all weights.

Q31. In distributed deep learning, what is the core concept of 'Data Parallelism'?

- ▢ **(a)** Partitioning the neural network layers across different nodes so each node holds 2 layers.
- ▢ **(b)** Replicating the entire model on every worker, assigning a distinct subset (shard) of the training dataset to each worker, and synchronizing gradients across workers at each step.
- ▢ **(c)** Running different machine learning frameworks on different servers.
- ▢ **(d)** Storing all dataset samples in a single SQLite database file.

Q32. When is 'Model Parallelism' (Tensor Parallelism or Pipeline Parallelism) strictly required instead of pure Data Parallelism?

- ▢ **(a)** When the dataset has fewer than 100 images.
- ▢ **(b)** When a single model's parameters and activation memory exceed the physical memory capacity of an individual accelerator.
- ▢ **(c)** When training on a laptop with only one CPU core.

- ▢ **(d)** When the learning rate is smaller than 0.001.

Q33. In the MPI standard, what does the collective operation 'MPI_Bcast' accomplish?

- ▢ **(a)** It sums arrays from all processes and stores the result on all processes.
- ▢ **(b)** It distributes equal chunks of a large array from a root process to all processes in the communicator.
- ▢ **(c)** It broadcasts an identical buffer from a designated root process to all other processes in the communicator.
- ▢ **(d)** It shuts down all worker nodes immediately.

Q34. What is the key functional difference between 'Reduce' and 'AllReduce' collective operations?

- ▢ **(a)** Reduce performs addition, while AllReduce performs subtraction.
- ▢ **(b)** Reduce combines data across all processes using an operator (e.g., SUM) and delivers the result only to a single root rank; AllReduce delivers the final reduced result to ALL processes.
- ▢ **(c)** AllReduce only works on single-node computers.
- ▢ **(d)** Reduce runs in parallel, whereas AllReduce runs serially.

Q35. What is the primary challenge of Asynchronous SGD (Async SGD) compared to Synchronous SGD in distributed training?

- ▢ **(a)** Async SGD cannot run on multiple machines.
- ▢ **(b)** Workers apply parameter updates based on stale weights (the 'stale gradient problem'), which destabilizes convergence and degrades final model accuracy.
- ▢ **(c)** Async SGD requires all workers to pause and wait for the slowest worker at every step.
- ▢ **(d)** Async SGD requires 100 times more network bandwidth than Synchronous SGD.

Q36. What is the 'straggler problem' in synchronous distributed deep learning training?

- ▢ **(a)** A bug where the compiler drops trailing zeros from floating point numbers.
- ▢ **(b)** A single slow worker node (due to thermal throttling, hardware faults, or I/O stalls) delays the entire cluster because all other nodes must wait for it at the synchronization barrier.
- ▢ **(c)** Network packets arriving out of order over TCP.
- ▢ **(d)** Images having mismatched dimensions in the training set.

Q37. Why is gradient bucketing (or gradient fusion) utilized in distributed training frameworks like Horovod and PyTorch DDP?

- ▢ **(a)** To reduce network overhead by aggregating thousands of small layer gradient tensors into a few large contiguous memory buffers before launching AllReduce.
- ▢ **(b)** To sort the gradients in alphabetical order.
- ▢ **(c)** To convert the neural network into an image classification model.
- ▢ **(d)** To encrypt model weights before transmission.

Q38. What is the difference between latency and bandwidth in a high-speed HPC network fabric?

- ▢ **(a)** Latency is the maximum volume of data transferred per second; bandwidth is the time taken for a bit to travel.
- ▢ **(b)** Latency is the round-trip transit time for a small message (e.g., microseconds); bandwidth is the sustained data transfer rate (e.g., Gbps or GB/s).

- ▢ **(c)** Latency and bandwidth are identical physical quantities measured in Hertz.
- ▢ **(d)** Latency applies only to wireless networks, while bandwidth applies only to fiber optics.

Q39. What is Remote Direct Memory Access (RDMA), and why is it essential for distributed deep learning over InfiniBand or RoCE?

- ▢ **(a)** A technique to remotely reboot hard drives over the internet.
- ▢ **(b)** A hardware capability allowing network adapters to transfer data directly between the memories of two remote servers without involving either server's OS kernel or CPU.
- ▢ **(c)** An unmanaged protocol that converts IP packets into analog television signals.
- ▢ **(d)** A cloud backup service for Python source code.

Q40. In a distributed cluster of 8 nodes with 8 GPUs each, what is the total 'world size' in Horovod or MPI terms?

- ▢ **(a)** 8
- ▢ **(b)** 16
- ▢ **(c)** 64
- ▢ **(d)** 512

Q41. What is the primary role of a Workload Manager (Job Scheduler) such as Slurm in an enterprise HPC cluster?

- ▢ **(a)** To replace Linux operating system kernels with Windows.
- ▢ **(b)** To arbitrate access to shared compute resources among competing users, queue jobs, enforce fair-share policies, and monitor execution across compute nodes.
- ▢ **(c)** To edit Python code and fix syntax errors automatically.
- ▢ **(d)** To manage physical power cabling in the server racks.

Q42. In a Slurm cluster, what is the role of the 'slurmd' daemon running on each compute node?

- ▢ **(a)** It manages the central accounting database for all billing.
- ▢ **(b)** It listens for job allocation orders from 'slurmctld', launches user tasks, monitors their CPU/memory consumption, and signals job completion.
- ▢ **(c)** It compiles TensorFlow from source code.
- ▢ **(d)** It provides a web GUI for submitting batch jobs.

Q43. Which command is used to submit a non-interactive batch script to the Slurm workload manager?

- ▢ **(a)** srun --interactive
- ▢ **(b)** sbatch job_script.slurm
- ▢ **(c)** scancel all
- ▢ **(d)** sinfo --kill

Q44. What is the difference between 'srun' and 'sbatch' in Slurm?

- ▢ **(a)** 'sbatch' submits a job to the batch queue and returns immediately; 'srun' launches job steps interactively or concurrently and blocks until tasks finish.
- ▢ **(b)** 'sbatch' only runs Fortran code, while 'srun' only runs Python.
- ▢ **(c)** 'srun' is used to delete jobs from the queue.

- ▢ **(d)** 'sbatch' can only run on a single CPU core.

Q45. Which Slurm directive specifies the maximum allowable runtime for a job before it is terminated by the scheduler?

- ▢ **(a)** #SBATCH --mem=32G
- ▢ **(b)** #SBATCH --time=02:00:00
- ▢ **(c)** #SBATCH --partition=compute
- ▢ **(d)** #SBATCH --output=slurm-%j.out

Q46. What does the symbol '%j' represent in the Slurm directive '#SBATCH --output=training_%j.log'?

- ▢ **(a)** The user's username.
- ▢ **(b)** The name of the partition.
- ▢ **(c)** The unique numerical Job ID allocated by Slurm.
- ▢ **(d)** The current date and time.

Q47. What does the command 'sinfo' display in a Slurm cluster environment?

- ▢ **(a)** The real-time CPU temperature of the head node.
- ▢ **(b)** The status of partitions and compute nodes in the cluster (e.g., idle, allocated, draining, down).
- ▢ **(c)** The remaining storage quota in the user's home directory.
- ▢ **(d)** The git commit history of the repository.

Q48. Why is training deep learning models on a shared Network File System (NFS) containing millions of tiny image files discouraged on HPC clusters?

- ▢ **(a)** Because NFS does not support JPEG or PNG images.
- ▢ **(b)** Because millions of tiny file open/read metadata operations overwhelm the centralized NFS metadata server, causing massive I/O stalls and bringing down filesystem responsiveness for all users.
- ▢ **(c)** Because NFS requires all files to be smaller than 10 bytes.
- ▢ **(d)** Because NFS automatically deletes files after 24 hours.

Q49. What is the purpose of the 'Environment Modules' system (`module load ...`) widely deployed on HPC clusters?

- ▢ **(a)** To dynamically adjust the terminal screen brightness.
- ▢ **(b)** To dynamically modify the user's environment (PATH, LD_LIBRARY_PATH, MANPATH) to load specific compiler versions, MPI runtimes, and software packages without version conflicts.
- ▢ **(c)** To reboot individual compute nodes without administrator permissions.
- ▢ **(d)** To encrypt Python scripts before execution.

Q50. If a user needs to cancel a queued or running Slurm job with Job ID 89452, which command should they execute?

- ▢ **(a)** sbatch --delete 89452
- ▢ **(b)** scancel 89452
- ▢ **(c)** squeue --kill 89452

- ▢ **(d)** sinfo --remove 89452